



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра системного программирования

Егоров Илья Георгиевич
группа 627

Суперкомпьютерное моделирование и технологии
**Гибридная MPI/OpenMP многопоточная реализация
решения гиперболического уравнения в частных производных
на трехмерном прямоугольном параллелепипеде**

ОТЧЁТ

Москва, 2025, 23 Октября

Содержание

1	Описание задания и программной реализации	3
1.1	Постановка задачи	3
1.2	Численный метод решения задачи	3
1.3	Программная реализация	4
2	Исследование производительности	6
2.1	Характеристики вычислительной системы	6
2.1.1	Параллельное ускорение MPI	6
2.1.2	Параллельное ускорение OpenMP	7

1 Описание задания и программной реализации

1.1 Постановка задачи

В трёхмерной замкнутой области $\Omega = [0, L_x] \times [0, L_y] \times [0, L_z]$ для $t \in [0, T]$ требуется найти решение $u(x, y, z, t)$ задачи

$$\begin{cases} u_{tt}(x, y, z, t) = a^2 \Delta u(x, y, z, t) \\ u(x, y, z, 0) = \phi(x, y, z) \\ u_t(x, y, z, 0) = 0 \\ u(0, y, z, t) = u(L_x, y, z, t) = 0 \\ u(x, 0, z, t) = u(x, L_y, z, t) \\ u(x, y, 0, t) = u(x, y, L_z, t) = 0 \end{cases} \quad (1)$$

где $u(x, y, z, t) = \sin \frac{\pi x}{L_x} \sin \frac{2\pi y}{L_y} \sin \frac{3\pi z}{L_z} \cos(a_t t)$, $a_t = \frac{\pi}{2} \sqrt{\frac{1}{L_x} + \frac{4}{L_y} + \frac{9}{L_z}}$ — точное аналитическое решение, $\phi(x, y, z) = u(x, y, z, 0) = \sin \frac{\pi x}{L_x} \sin \frac{2\pi y}{L_y} \sin \frac{3\pi z}{L_z}$ и $a^2 = \frac{1}{4}$.

1.2 Численный метод решения задачи

Пусть заданы $N, K \in \mathbb{N}$, тогда пусть $h_x = \frac{L_x}{N}$, $h_y = \frac{L_y}{N}$, $h_z = \frac{L_z}{N}$, $\tau = \frac{T}{K}$ и $\omega_{h\tau} = \overline{\omega_h} \times \omega_\tau$, где

$$\overline{\omega_h} = \{(x_i, y_j, z_k) | x_i = ih_x, y_j = jh_y, z_k = kh_z, i, j, k \in \overline{0, N}\}$$

$$\omega_\tau = \{t = n\tau, n \in \overline{0, K}\}$$

Пусть $\omega_h = \text{int} \overline{\omega_h}$, т.е. внутренние узлы сетки.

Пусть $u_{ijk}^n = u(x_i, y_j, z_k, t_n)$, $\phi_{ijk} = \phi(x_i, y_j, z_k)$

Тогда

$$\begin{aligned} u_{0jk}^n &= u_{Njk}^n = u_{ij0}^n = u_{ijN}^n = 0, & i, j, k \in \overline{0, N}, n \in \overline{0, K} \\ u_{i0k}^n &= u_{iNk}^n, u_{i1k}^n = u_{iN+1k}^n, & i, k \in \overline{0, N}, n \in \overline{0, K} \\ u_{ijk}^0 &= \phi_{ijk} = \phi_{ijk} = \phi(x_i, y_j, z_k), & (x_i, y_j, z_k) \in \omega_h \\ u_{ijk}^1 &= u_{ijk}^0 + \frac{a^2 \tau^2}{2} \Delta_h \phi_{ijk} & (x_i, y_j, z_k) \in \omega_h \\ u_{ijk}^{n+1} &= 2u_{ijk}^n - u_{ijk}^{n-1} + \Delta_h u_{ijk}^n & (x_i, y_j, z_k) \in \omega_h \end{aligned}$$

Здесь Δ_h — семиточечный разностный аналог оператора Лапласа:

$$\Delta_h u_{ijk}^n = \frac{u_{i+1jk}^n - 2u_{ijk}^n + u_{i-1jk}^n}{h_x^2} + \frac{u_{ij+1k}^n - 2u_{ijk}^n + u_{ij-1k}^n}{h_y^2} + \frac{u_{ijk+1}^n - 2u_{ijk}^n + u_{ijk-1}^n}{h_z^2}$$

Будем рассматривать случай, когда $L_x = L_y = L_z = L$ и, соответственно, $h_x = h_y = h_z = h$. Для устойчивости решения требуется

$$\tau < h$$

Или же

$$\gamma = \frac{\tau}{h} < 1$$

Тогда при заданных L, N, K можно положить $T = \tau K = \gamma h K = \frac{\gamma L K}{N}$.

1.3 Программная реализация

Программа реализована на языке C++ (с++11/с++20). Для записи решения написан шаблонный класс *Grid4D*, который абстрагирует взаимодействие с памятью и индексированием. Также написаны функциональные классы *AnalyticalFunction* для инкапсуляции вычисления значения аналитической функции, *AnalyticalFunctionGrid* — обёртка, делающая то же самое, но в терминах узлов сетки, и *InitialFunctionGrid* — для вычисления начальных условий. Для непосредственного решения написан класс *Solver* с методом *solve*, возвращающий решение, — объект класса *Grid4D*, значение погрешности и времени, затраченного на вычисление решения и ошибки.

При реализации функционала параллелизма с распределённой памятью были добавлены классы *Communicator* и *CubeCommunicator*, вспомогательные перечислимые типы *Edge* и *Tag* и производные от них, упрощающие межпроцессорное взаимодействие, а также класс *SolverFactory* для более удобного создания объектов класса *Solver* для конкретного процесса с его собственными параметрами. Также была использована асинхронная оптимизация — обмен данными и вычисления происходят одновременно: сначала инициализируется неблокирующий обмен гало и интерфейсными узлами, затем происходят вычисления на внутренних узлах, после чего процессы проходят барьер на обмен и проводят вычисления на интерфейсных узлах.

В качестве аргументов программа получает на вход N и T — количество потоков OpenMP, а также px , py и pz — параметры разбиения сетки по процессам вдоль соответствующих осей. Значение K согласно условиям равно 20, а L — 1 и π .

2 Исследование производительности

2.1 Характеристики вычислительной системы

Имя: ПВС «IBM Polus»

Пиковая производительность: 55.84 TFlop/s

Производительность (Linpack): 40.39 TFlop/s

Вычислительных узлов: 5

На каждом узле:

Процессоры IBM Power 8: 2

NVIDIA Tesla P100: 2

Число процессорных ядер: 20

Число потоков на ядро: 8

Оперативная память: 256 Гбайт (1024 Гбайт узел 5)

Коммуникационная сеть: Infiniband / 100 Gb

Система хранения данных: GPFS

Операционная система: Linux Red Hat 7.5

2.1.1 Параллельное ускорение MPI

Дополнительно используемые опции компиляции: *-std=c++11 -O3 -fopenmp*.

На табл. 1 и 2 и рис. 1 и 2 хорошо видно наличие параллельного ускорения. При увеличении числа MPI процессов до 16 включительно виден линейный характер уменьшения затраченного времени, при больших значениях рост уменьшения времени затухает.

Process Number	Node Number per Side	Solve Time (sec)	Acceleration	Error
1	128	7.419	1.000	3.13E-05
2	128	3.876	1.914	3.13E-05
4	128	1.951	3.802	3.13E-05
8	128	1.030	7.206	3.13E-05
16	128	0.555	13.370	3.13E-05
1	256	56.214	1.000	2.01E-06
2	256	29.179	1.927	2.01E-06
4	256	14.786	3.802	2.01E-06
8	256	7.753	7.250	2.01E-06
16	256	4.149	13.550	2.01E-06

Таблица 1: Зависимость времени решения от числа процессов для $L = 1$

Process Number	Node Number per Side	Solve Time (sec)	Acceleration	Error
1	128	7.414	1.000	3.13E-05
2	128	3.876	1.913	3.13E-05
4	128	1.953	3.795	3.13E-05
8	128	1.029	7.204	3.13E-05
16	128	0.558	13.298	3.13E-05
1	256	56.202	1.000	2.01E-06
2	256	29.157	1.928	2.01E-06
4	256	14.785	3.801	2.01E-06
8	256	7.756	7.246	2.01E-06
16	256	4.156	13.522	2.01E-06

Таблица 2: Зависимость времени решения от числа процессов для $L = \pi$

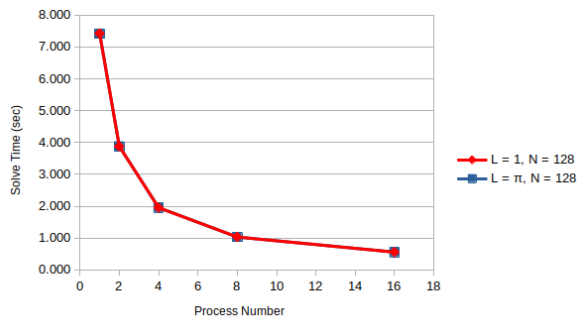


Рис. 1: Зависимость времени решения от числа процессов для $N = 128$

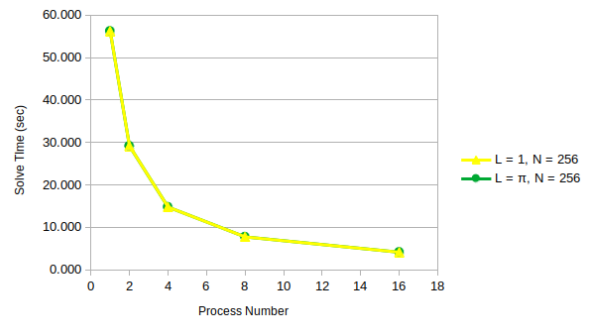


Рис. 2: Зависимость времени решения от числа процессов для $N = 256$

2.1.2 Параллельное ускорение OpenMP

На табл. 3 и 4 и рис. 3 и 4 показано время вычислений в зависимости от числа нитей для каждого MPI процесса. При измерениях запускалось 8 MPI процессов, т.е. $px = py = pz = 2$. Число потоков увеличивается до 8 включительно, так как каждое ядро "Полюса" имеет ровно 8 потоков.

Thread Number	Node Number per Side	Solve Time (sec)	Acceleration	Error
1	128	1.318	1.000	3.13E-05
2	128	1.096	1.202	3.13E-05
4	128	0.860	1.532	3.13E-05
8	128	0.583	2.259	3.13E-05
1	256	10.338	1.000	2.01E-06
2	256	6.795	1.521	2.01E-06
4	256	4.621	2.237	2.01E-06
8	256	3.634	2.845	2.01E-06

Таблица 3: Зависимость времени решения от числа нитей процессов для $L = 1$

Thread Number	Node Number per Side	Solve Time (sec)	Acceleration	Error
1	128	1.322	1.000	3.13E-05
2	128	1.028	1.287	3.13E-05
4	128	0.821	1.611	3.13E-05
8	128	0.582	2.271	3.13E-05
1	256	9.833	1.000	2.01E-06
2	256	6.262	1.570	2.01E-06
4	256	4.401	2.234	2.01E-06
8	256	3.706	2.653	2.01E-06

Таблица 4: Зависимость времени решения от числа нитей процессов для $L = \pi$

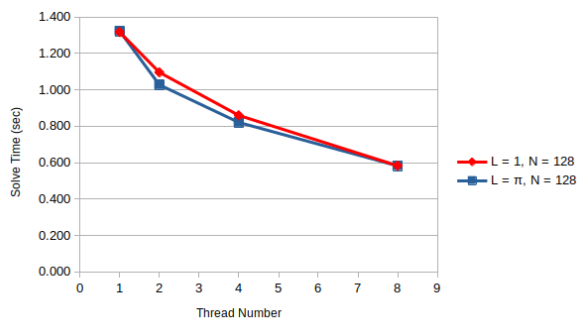


Рис. 3: Зависимость времени решения от числа нитей процессов для $N = 128$

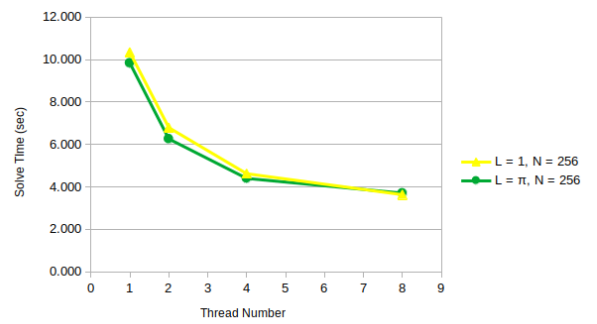


Рис. 4: Зависимость времени решения от числа нитей процессов для $N = 256$